

C Programming Assignment

Pattern Menu Using Switch Statement

Nested loops | Separate parameterless functions | No arrays

Aim

To write a C program that displays a menu and uses a switch statement to print four basic square/rectangular patterns selected by the user. The patterns are generated using nested loops without arrays, and each pattern is implemented in a separate parameterless function with no return value.

Algorithm

1. Start the program.
2. Display a menu with four pattern choices: solid square, solid rectangle, row-number pattern, and column-number pattern.
3. Read the user's choice.
4. Use a switch statement to call the corresponding parameterless function.
5. Inside the selected function, read the required number of rows and/or columns.
6. Use nested loops to generate and print the required pattern.
7. If the choice is not 1 to 4, display an invalid-choice message.
8. Stop the program.

C Program

```
#include <stdio.h>

void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main(void)
{
    int choice;

    printf("PATTERN MENU\n");
    printf("1. Solid square pattern\n");
    printf("2. Solid rectangular pattern\n");
    printf("3. Row-number pattern\n");
    printf("4. Column-number pattern\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
```

```

{
    case 1:
        solidSquare();
        break;
    case 2:
        solidRectangle();
        break;
    case 3:
        rowNumberPattern();
        break;
    case 4:
        columnNumberPattern();
        break;
    default:
        printf("Invalid choice!\n");
}

return 0;
}

void solidSquare(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void solidRectangle(void)
{
    int m, n, i, j;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void rowNumberPattern(void)
{
    int m, n, i, j;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
            printf("%d ", i);
        printf("\n");
    }
}

```

```

void columnNumberPattern(void)
{
    int m, n, i, j;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
            printf("%d ", j);
        printf("\n");
    }
}

```

Prepared Test Cases

Test Case	Choice	Input	Expected Output
1	1	n = 4	4 x 4 solid square of *
2	2	m = 3, n = 5	3 x 5 solid rectangle of *
3	3	m = 4, n = 4	Rows: 1 1 1 1 through 4 4 4 4
4	4	m = 4, n = 4	Each row: 1 2 3 4
5	7	-	Invalid choice!

Expected Outputs

1. Solid Square Pattern (n = 4)

```

* * * *
* * * *
* * * *
* * * *

```

2. Solid Rectangular Pattern (m = 3, n = 5)

```

* * * * *
* * * * *
* * * * *

```

3. Row-Number Pattern (m = 4, n = 4)

```

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

```

4. Column-Number Pattern (m = 4, n = 4)

```
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

Result

The C program was designed to display a menu and generate the selected square/rectangular pattern using a switch statement, nested loops, and separate parameterless functions without arrays. The prepared test cases cover all four valid menu options and an invalid choice.

Execution Screenshots

Compilation of the C program

```
(kali㉿kali)-[~/Desktop/apple]
$ gcc hdoc23.c -o hdoc23

(kali㉿kali)-[~/Desktop/apple]
$
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS

Test Case 1 - Solid square pattern (choice 1, n = 4)

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc23
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 1
Enter number of rows: 4
* * * *
* * * *
* * * *
* * * *
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS

Test Case 2 - Solid rectangular pattern (choice 2, rows = 3, columns = 5)

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc23
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 2
Enter number of rows and columns: 3 5
* * * * *
* * * * *
* * * * *
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS

Test Case 3 - Row-number pattern (choice 3, rows = 4, columns = 4)

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc23
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 3
Enter number of rows and columns: 4 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS

Test Case 4 - Column-number pattern (choice 4, rows = 4, columns = 4)

```
(kali@kali)-[~/Desktop/apple]
$ ./hdoc23
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 4
Enter number of rows and columns: 4 4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS

Invalid choice test (choice 7)

```
(kali@kali)-[~/Desktop/apple]
$ ./hdoc23
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 7
Invalid choice!
```

Actual Output: Matches the expected output shown for this test case.

Result: PASS